5101 Assignment 3 - Development and Documentation (2607 v3.4)

100 Points Possible

8/11/2026

Attempt 1

In Progress

NEXT UP: Submit Assignment

Add Comment

Unlimited Attempts Allowed

9/11/2026

Details

Student Declaration

By submitting this assessment, I acknowledge that:

- I have read and understood this assessment fully including the sections **HOW TO SUBMIT A** (https://icled.instructure.com/courses/2547/assignments/$CANVAS_COURSE_REFERENCE$/mo and the **ICL ASSESSMENT INSTRUCTIONS** (https://icled.instructure.com/courses/2547/assignments/$CANVAS_COURSE_REFERENCE$/mo .
- I understand that **my submission to Canvas is my official submission** and that handing in count as an official submission.
- I confirm that all work completed for this assessment is my own work and has been produce
- I confirm that this assignment has not, in whole or in part, been submitted for assessment in Graduate Business School, or at any other educational institution.

AI Code of Honour

I promise to adhere to the following code of honour in the use of Artificial Intelligence (AI) techno assessments:

- I undertake not to use AI in an inauthentic way to boost my performance in academic assess
- I will seek clarification from my instructors or academic advisors if I have any doubts or conc technologies.
- If I have utilised AI tools to enhance my learning or productivity, I will acknowledge it by provi 7th format, to highlight the specific areas where AI was employed.
- I will not submit work that has been written in another language and translated, generated, re
- When required, I should be able to provide evidence of authorship, including but not limited t
- I understand that by adhering to this code of honour, I am not only preserving my own acade and value of the degrees and qualifications awarded by ICL Graduate Business School.

Assignment 3 - Development and Documentation

v3.4 (based on final draft 3); updated: 09/10/2026; 2607 term

Weighting: 40% of final grade
Total Marks: 100
Assessment Type: Individual assignment
Learning Resources: [5101 Assignment 3 - C# Windows Forms and LocalDB Learning Resources (https://icled.instructure.com/courses/3059/pages/5101-assignment-3-c-number-windows-forms-and-localdb-learning-resources)]
Due Date/Time: See Canvas LMS setting for due dates/times
Submission:

1. Structured report (DOCX format), including supporting screenshots and relevant code excerpts.
2. Complete Visual Studio C# solution/project folder (ZIP format), including the required LocalDB database files.
3. Individual application demonstration (live or recorded, as instructed by the lecturer).

Assignment Overview

This individual assignment requires you to independently design, implement, test, debug and document a basic object-oriented Windows Forms application based on the business scenario provided in the **Assignment 3 Case Briefing**.

You will apply software engineering and programming principles introduced in this course to:

- analyse the business problem;
- design an appropriate class structure;
- implement the solution using C# and Windows Forms;
- validate user input and enforce business rules;
- test and debug your application; and
- document your design and development decisions.

All planning, design, implementation and testing must address the supplied business scenario.

The primary focus of this assignment is on:

- correct use of object-oriented principles (classes, encapsulation, methods and properties);
- functional working software;
- logical separation between user interface and business logic;

- input validation and error handling; and
- systematic testing and debugging.

While documentation is required, the core emphasis of this assessment is on producing a working, well-structured application.

Learning Outcomes (LOs) Assessed and Weighting

This individual assignment assesses the following LOs from *BBIS 5101 Introduction to Object Oriented Programming*:

- **LO #2:** Apply software engineering principles to analyse and solve problems related to business.
- **LO #3:** Design and implement, including testing and debugging, basic object-oriented programs as they apply to business using a contemporary programming language.

The assignment is weighted at 40% of the overall course and is marked out of 100 marks. The pass mark for this assignment is 50/100 marks (50%; C- grade).

Case Briefing

You must develop a C# Windows Forms application based on the business scenario provided in:

2607-5101 Assignment 3 Case Briefing - Campus Facilities Issue Register (https://icled.instructure.com/courses/3059/pages/2607-5101-assignment-3-case-briefing-campus-facilities-issue-register)

The Case Briefing provides the business context, functional requirements, information to be managed, suggested business rules and scope. Your application must address the supplied business scenario and meet the requirements specified in this assessment brief. Do not introduce major additional functionality or substantially change the supplied scenario without lecturer approval.

Minimum Application Requirements

Your application must:

- Use **C# and Windows Forms (WinForms)** for the user interface, unless written approval is given for an alternative UI framework;
- Include **Add (Create), View (Read), Edit (Update) and Delete** functionality;
- Connect to a **SQL Server LocalDB database (.mdf)** using basic **ADO.NET**, with one main

database table;
- Provide at least **one working search or filter feature;**
- Use **at least two interacting custom C# classes** (excluding Form classes) with meaningful responsibilities;
- Demonstrate **encapsulation**, including private fields and appropriate public properties or methods;
- Separate business logic from user interface code where appropriate;
- Implement **basic input validation** (e.g., required fields, correct data types, valid ranges or dates);
- Enforce **at least three business rules**, using those suggested in the Case Briefing or other appropriate rules;
- Demonstrate appropriate testing and debugging.

Advanced architectural patterns are not required. The focus is on correct implementation of foundational object-oriented programming concepts.

Evidence Requirement (Important)

You must provide clear evidence that the application requirements have been implemented. Evidence may include:

- Screenshots showing the application running (CRUD operations, validation messages, search/filter results and data updates);
- Short, annotated code excerpts demonstrating key implementation points (e.g., validation logic, SQL commands and class methods);
- Testing and debugging evidence as required in Task 3.

Simply stating that a feature exists is not sufficient. Evidence must be included in your written report and be consistent with your submitted application.

Assignment Tasks and Deliverables

Task 1: Planning & Design Blueprint (20 marks)

Before you begin coding, you must plan your application carefully.

This task assesses your ability to apply basic software engineering principles to analyse the business problem and design a clear, implementable solution.

Your planning must directly support the system you will build in Task 2. Keep your work concise, practical, and logically structured.

Recommended length: 5–8 pages, including tables and diagrams.

Task 1.1 – Business Purpose & Functional Requirements (5 marks)

A. Business Context (short explanation)

In 5–8 sentences, explain:

- The business problem being addressed;
- The primary user of the system;
- The overall purpose of the application;
- How the system supports or improves the business process.

Focus on clarity and practical relevance.

B. Functional Requirements

Provide a structured list of features your system will include. You must include:

- Add (Create);
- View (Read);
- Update;
- Delete;
- At least one Search or Filter feature.

C. Validation / Business Rules

Define **at least three validation or business rules**.

- Each rule must be written in this format: **Rule # – Field – Condition – User Message**
- Example: Rule 1 – Quantity – Must be greater than 0 – “Quantity must be a positive number.”

Briefly state how the rule will be enforced (e.g., conditional statement, TryParse, range check).

Note: Only basic validation logic is required. Advanced exception architecture is not required.

Task 1.2 – Data Design (5 marks)

Design the main business entity and corresponding database table for your application, based on the supplied Case Briefing.

Provide a table including:

- Field Name;
- C# Data Type;
- Database Type (e.g., nvarchar, int, decimal);
- Required? (Yes/No);
- Example Value.

Requirements:

- Include **6–10 fields;**
- Clearly identify the **Primary Key;**
- State whether the Primary Key will be auto-generated;
- Ensure fields align with your functional requirements.

Note: Only a single main entity is required. Complex relational modelling is not required.

Task 1.3 – Class Design (5 marks)

Provide a simple class design.

You must include:

- At least two interacting custom classes (excluding Form classes), each with a meaningful responsibility;
- Appropriate fields and properties demonstrating basic encapsulation;
- Key methods (e.g., Save, Update, Calculate, Validate).

A simple hand-drawn or digital diagram is acceptable.

Briefly explain the responsibility of each class (2–4 bullet points per class).

Note: Inheritance, interfaces, generics, recursion, and advanced design patterns are not required for this assignment.

Task 1.4 – User Interface Planning (5 marks)

A. Main Form Sketch

Provide a labelled sketch of your main form showing:

- Input controls;
- CRUD buttons;
- DataGridView for displaying records;
- Search/filter controls.

A hand-drawn sketch is acceptable if clear and readable.

B. Control Mapping Table

Provide a table including:

- Control Type (TextBox, ComboBox, Button, DataGridView, etc.);
- Control Name (use proper naming conventions, e.g., txtName, btnAdd);
- Linked Data Field;
- Purpose.

Your UI design must be logically consistent with your data design and validation rules.

Important Notes

- Keep your work clear and structured;
- Planning must directly support your implementation in Task 2;
- Advanced object-oriented concepts such as inheritance hierarchies, interfaces, generics, recursion, or architectural layering are not required;
- The focus is on foundational OOP concepts: classes, encapsulation, methods, properties, and basic event-driven programming;
- Your final design must align with the implemented application. Briefly explain any significant design changes in Task 2.

Task 2: Application Implementation (50 marks)

In this task, you will implement the application you planned in Task 1 using **C# and Windows Forms**.

Your implementation must demonstrate correct use of foundational object-oriented programming concepts and basic database integration.

The focus is on producing a working, logically structured application.

Task 2.1 – Project Structure & Class Implementation (15 marks)

You must implement:

- At least two interacting custom classes (excluding Form classes), each with a meaningful responsibility;
- Appropriate fields and properties (encapsulation required);
- Logical methods (e.g., Save, Update, Validate, Calculate);
- Clear separation between business logic and UI logic where appropriate.

Requirements:

- Fields should be private;
- Access should be provided through properties or methods;
- Avoid placing all logic directly inside the Form.

Note: Advanced concepts such as inheritance hierarchies, interfaces, generics, or design patterns are not required.

Task 2.2 – Database Integration (15 marks)

You must use **SQL Server LocalDB** and basic **ADO.NET**.

Requirements:

- Create a simple `.mdf` database in Visual Studio;
- Create one main database table consistent with your Task 1 data design and the supplied Case Briefing;
- Implement CRUD operations:
  - Insert
  - Select
  - Update
  - Delete
- Display records in a `DataGridView.`

You may write SQL queries directly in your form code. Formal data access layers are not required.

Note: Advanced database architecture, repository patterns, or multi-layer separation are not required.

Task 2.3 – Functional Features & Validation (10 marks)

Your application must:

- Fully implement all planned CRUD operations;
- Implement at least one search or filter feature;
- Enforce at least three validation rules defined in Task 1;
- Display meaningful error messages for invalid input.

Validation must occur before executing SQL commands.

Basic conditional checks and TryParse methods are sufficient.

Task 2.4 – User Interface & Usability (10 marks)

Your application must:

- Use appropriate WinForms controls;
- Follow consistent naming conventions (e.g., txtName, btnAdd);
- Provide a clear and usable layout;
- Display updated data correctly after operations;
- Clear input fields appropriately when needed.

Event handling must function correctly.

Note: The focus is on clarity and correct functionality rather than advanced visual styling.

Technical Expectations

Your application must:

- Compile successfully and run without errors that prevent the required functionality from working;
- Connect successfully to the LocalDB database;
- Perform all required CRUD operations;
- Use foundational OOP concepts (classes, properties, methods, encapsulation);
- Demonstrate event-driven programming.

You are not required to implement:

- Inheritance hierarchies;
- Interfaces;
- Generics;
- Recursion;
- File handling;
- Multi-layer architecture;
- Advanced exception frameworks.

This assignment focuses on correct implementation of foundational OOP principles.

Task 3: Testing & Debugging (15 marks)

In this task, you must demonstrate that your application has been properly tested and that you understand how debugging improves software quality.

Testing must be practical and directly related to your implemented system. Formal testing frameworks are not required.

Task 3.1 – Test Case Table (5 marks)

You must provide a structured test case table with **a minimum of eight (8) test cases**.

Your test cases must include:

- At least **2 valid input tests;**
- At least **2 invalid input tests;**
- At least **1 boundary test case.**

Each test case must include:

- Test Case ID;
- Input Values;
- Expected Result;

- Actual Result;
- Pass / Fail.

Your testing must cover all four CRUD operations, the search/filter feature and relevant validation rules defined in Task 1.

Note: Eight test cases is the minimum. Include additional tests as needed to achieve complete coverage. Report actual results, including any failures.

Task 3.2 – Validation & Error Handling Evidence (5 marks)

You must provide evidence that your validation rules are working.

Include:

- At least **two screenshots** showing validation messages;
- A short explanation of how the validation is implemented (e.g., conditional checks, `TryParse`, range validation);
- Supporting code excerpts or test results, where necessary, to demonstrate that **at least three validation/business rules** are enforced.

Validation must occur before executing database operations. Advanced exception handling architecture is not required.

Task 3.3 – Debugging Example (5 marks)

You must describe one real issue (bug) encountered during development.

Explain:

- What the problem was;
- What caused it;
- How you identified it;
- How you fixed it.

This should be concise (approximately 150–250 words).

The bug must be genuine and related to your implementation (e.g., logic error, incorrect SQL statement, null reference, incorrect event handling).

Important Notes

- This task assesses your ability to test and debug your own code;
- You are not required to use automated testing tools;
- Unit testing frameworks are not required;
- The focus is on structured manual testing and basic debugging skills.

Technical Expectation

Your testing must demonstrate that:

- CRUD operations work correctly;
- Validation rules are enforced;
- Errors are handled clearly;
- The system behaves as expected under different input conditions.

Task 4: Demonstration of Application (15 marks)

In this task, you must demonstrate your completed application and show that you understand how your system works.

This task verifies that:

- Your application functions correctly;
- You understand your own code;
- You can explain your design and implementation decisions.

The demonstration may be conducted **live in class** or submitted as a **screen-recorded video**, as instructed by your lecturer.

4.1 Application Demonstration (10 marks)

You must demonstrate the following:

- Successful launch of the application;
- Working database connection;
- Add (Create) operation;
- View (Read) operation;
- Update operation;
- Delete operation;
- Working search or filter feature;
- At least one validation rule in action.

You must clearly show the system performing these operations.

All demonstrated features must match what was planned in Task 1.

4.2 Code Explanation and Understanding (5 marks)

You must explain key parts of your system, including:

- The role of your main custom classes;
- How encapsulation is used (fields and properties);

- How CRUD operations are implemented;
- How validation is enforced;
- How the data storage connection works (basic explanation only).

If submitting a recorded video, you must clearly narrate your explanation. Regardless of the demonstration format, the lecturer may require a follow-up verification, including questions about your code or a minor modification to your application.

You are expected to demonstrate understanding of your own work, not memorisation.

Demonstration (Delivery) Requirements

Duration: **Between 8–10 minutes** for either live or recorded demonstrations, excluding any follow-up assessor questions.

If the demonstration is recorded:

- Include screen capture of the running application and clear narration;
- Demonstrate the required application features.

If the demonstration is live:

- Be prepared to answer questions about your code;
- You may be asked to explain or make a minor modification to part of your application.

Important Notes

- This task verifies individual ownership of the work;
- You must be able to explain your implementation clearly;
- Advanced architectural discussion is not required;
- The focus is on foundational OOP understanding and practical implementation.

⚠️ ***Note:*** *Failure to complete a demonstration may result in the loss of up to 15 marks.*

Marking Summary (100 marks total)

Task	Description	Marks
Task 1	Planning & Design Blueprint	20
Task 2	Application Implementation (OOP, CRUD, DB, UI)	50
Task 3	Testing & Debugging	15
Task 4	Demonstration of Application	15
Total		**100**

This assignment is marked in 0.5 mark increments within each rubric criterion band. The mark awarded depends on how fully your submitted assignment work meets the stated criteria.

Submission Requirements

Submit the following assessment items through Canvas **on or before the specified due date and time**, unless alternative submission or demonstration arrangements have been advised by the lecturer.

#1. A structured, well-organised, and professionally formatted report (DOCX format)

Report Structure

- Title page
  with programme name, course/paper name and number, assessment number, a report title, your full name and student ID.
- Student Declaration (Page)
- Table of Contents/Table of Figures
- **Task 1: Planning & Design Blueprint**
  - Business purpose and functional requirements
  - Validation rules
  - Data design table
  - Class design
  - UI planning and control mapping
- **Task 2: Application Implementation**
  - Explanation of project structure
  - Key code excerpts (not full code listings)
  - Screenshots showing:
    - CRUD functionality
    - Database integration and stored records
    - Working user interface
    - Search or filter functionality
    - Validation in action
- **Task 3: Testing & Debugging Report**
  - Testing
  - Debugging
- Appendix - containing additional supporting material (if any)

Important

- All explanations must clearly link to screenshots and/or code excerpts.
- Implementation claims must be verifiable.
- Screenshots must be readable and clearly labelled.
- Submissions that cannot be verified through evidence may receive reduced or no marks for the relevant section.
- Excessive length will not attract additional marks. Clarity and logical consistency are more important than volume.

#2. Zipped Visual Studio C# solution/project folder

The zipped folder must:

- Contain the complete Visual Studio solution and C# Windows Forms project, including the `.sln`, project and source files.
- Include the SQL Server LocalDB database (`.mdf`) and other required project files.
- Retain the database connection configuration so the application can access its database without requiring the assessor to recreate the connection or database structure.
- Match the implementation and evidence presented in the written report.
- Compile and run successfully, with the required CRUD, search/filter and validation features working.

Before submitting: Extract your ZIP file into a separate folder and confirm that the solution opens, builds and runs correctly, including its database connection.

Your report, application and demonstration will be assessed against the requirements and marking rubric for Tasks 1-4.

#3. Application Demonstration

Complete the individual application demonstration as required in Task 4. The lecturer will advise whether the demonstration is conducted live or submitted as a recorded video, including the relevant submission arrangements.

Your demonstration must use the application and code submitted for assessment.

File Naming Convention

5101_Assignment3_StudentID_FullName.docx
5101_Assignment3_StudentID_FullName.zip

Late Submissions

- A penalty of 10% of the total marks that can be awarded for the assessment will be deducted automatically for each day an assignment is late without an approved extension;
- Assessments submitted later than five calendar days after the due date without an approved extension will be regarded as ‘Did Not Complete’ (DNC) and receive zero for that

assessment;
- On Canvas LMS:
  - **Due** specifies the **specified due date and time** for the assessment;
  - **Until** specifies the **final date and time** that the submission link remains open;
    If an **Until** date/time is set, it is provided only to allow for late submissions. Submissions made after the **Due** date/time are considered **late submissions** and will incur late penalties, even if the submission link remains open.

Authenticity Verification (of Submitted Work)

As part of academic integrity and quality assurance processes, the lecturer/assessor may **select submissions for verification**, either randomly and/or where authenticity concerns arise.

When requested, you must explain your work, including the steps you took, key decisions made, and your understanding of the concepts, techniques, frameworks and tools used. You must keep **all draft and final versions** of your assessment work.

This process ensures that all submitted work is:

- your own work,
- demonstrates a clear understanding of the assessment tasks, and
- has not been copied, outsourced (completed by another individual or external service), or generated by unauthorised tools.

Failure to satisfactorily explain your work may result in referral for an academic integrity review in accordance with ICL's policies.

View Rubric

5101 A3 v3.4 (1)	
Criteria	**Ratings**
1.1 Business Purpose & Functional Requirements (5 marks)	

	Excellent	Good	Satisfactory	Poo
	Clear and realistic business context aligned to the selected scenario. Primary user and system purpose are clearly explained. Functional requirements include complete CRUD and at least one search/filter feature. At least three validation/business rules are specific, implementable, and correctly formatted. Strong logical alignment between problem and features.	Business context mostly clear and relevant. CRUD features included with minor omissions or limited detail. Validation rules present but may lack precision or formatting consistency. Overall alignment is mostly logical.	A basic explanation of the business problem is provided. CRUD requirements are listed but lack detail or structure. At least three validation rules are included, though some may be simplistic or loosely defined. The connection between the problem and features is present but limited.	Unclear or g business explanation. Incomplete (requirements Validation ru missing, inc or unrealistic alignment be scenario and planned feat
1.2 Data Design (5 marks)	**5 to >3.95 pts**	**3.95 to >3.2 pts**	**3.2 to >2.45 pts**	**2.45 to >**

	Excellent	Good	Satisfactory	Poo
	Well-structured data table with 6–10 appropriate fields. Correct C# and database data types selected. Primary key clearly identified and appropriate. Fields align strongly with functional requirements. Evidence of clear and logical thinking.	Data structure mostly appropriate. Minor inconsistencies in field selection or data types. Primary key identified. Alignment with functional requirements generally clear.	A basic table structure with 6–10 fields is presented. Most fields are relevant, though minor inconsistencies in naming or data types may be evident. A primary key is identified. The design generally supports the required functionality but lacks refinement.	Illogical or incomplete c structure. Inc or inappropr data types. F key undear missing. Wea Weak or unc linkage to requirements Minimal evid
1.3 Class Design & OOP Structure (5 marks)	At least two interacting custom classes are clearly planned. Encapsulation, fields, properties and methods are correctly identified. Class responsibilities and interactions are logical and clearly explained. Demonstrates sound foundational OOP understanding. **5 to >3.95 pts**	Two interacting classes are planned with mostly logical responsibilities, methods and encapsulation. Minor weaknesses in class interaction or responsibility separation. **3.95 to >3.2 pts**	Two custom classes are planned with basic responsibilities and methods. Encapsulation is attempted, and some interaction between classes is identified, though the design may lack clarity or consistency. **3.2 to >2.45 pts**	encapsulatic Responsibili poorly define unrealistic. **2.45 to >**
	5 to >3.95 pts	**3.95 to >3.2 pts**	**3.2 to >2.45 pts**	**2.45 to >**

1.4 UI Planning & Control Mapping (5 marks)	**Excellent** Clear, well-labelled UI sketch showing appropriate controls (CRUD, DataGridView, search/filter). Control mapping table is complete and technically accurate. Naming conventions are appropriate. Strong consistency with data design and requirements. **5 to >3.95 pts**	**Good** UI sketch mostly clear. Required controls present. Control mapping mostly accurate with minor omissions or inconsistencies. **3.95 to >3.2 pts**	**Satisfactory** A basic UI sketch is provided including main controls. The control mapping table is present but may contain small omissions or inconsistencies. The UI generally reflects the data structure but lacks clarity or precision. **3.2 to >2.45 pts**	**Poo** Unclear or p structured U Missing key controls. We incorrect ma table. **2.45 to >**
2.1 Project Structure & Class Implementation (15 marks)				

	Excellent	Good	Satisfactory	Poo
	At least two meaningful, interacting custom classes are implemented. Encapsulation is correctly applied. Class responsibilities and methods are logically structured, with appropriate separation of business and UI logic. Code is readable and well organised.	Two interacting custom classes are implemented with mostly appropriate responsibilities and encapsulation. Methods and separation of logic are generally sound, with minor weaknesses.	Two custom classes are implemented with basic responsibilities and some meaningful interaction. Encapsulation is partially applied, and some business logic may remain in the Form. Demonstrates basic OOP understanding.	Only minima structure implemented Encapsulatio or inconsiste Majority of lo placed inside Form. Limite understandi
2.2 Database Integration (15 marks)	Local database correctly created and connected. CRUD operations fully functional. SQL statements accurate and logically implemented. DataGridView correctly displays updated records. No major database errors.	Database connection works and most CRUD operations function correctly. Minor issues in update/refresh behaviour. Data generally reflected correctly in UI.	Database connection established. At least two CRUD operations function correctly. Some inconsistencies in data refresh or execution. Demonstrates basic integration.	Database pa responsibilit connected b separation. one CRUD operation wo reliably. Freq runtime issu inconsistent behaviour.
	15 to >11.85 pts	**11.85 to >9.6 pts**	**9.6 to >7.35 pts**	**7.35 to >**
	15 to >11.85 pts	**11.85 to >9.6 pts**	**9.6 to >7.35 pts**	**7.35 to >**

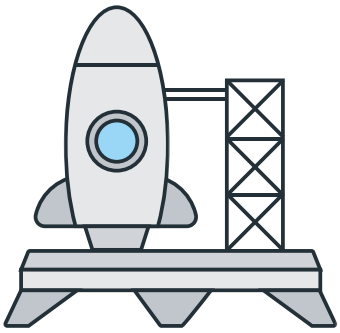
	Excellent	Good	Satisfactory	Poo

2.3 Functional Features & Validation (10 marks)	**Excellent** All planned CRUD features implemented and working. At least one search/filter feature implemented correctly. At least three validation rules enforced before database execution. Clear and meaningful error messages displayed.	**Good** Most CRUD features functional. Search/filter present but may have minor issues. Validation rules implemented but may lack robustness or clarity.	**Satisfactory** Basic CRUD functionality working. Validation rules implemented but limited or inconsistently applied. Search/filter may be basic or partially functional. Meets minimum functional requirement.	**Poo** Incomplete C implementat Validation w inconsistentl enforced. Lir functional re
	Excellent UI is clear, logical, and usable. Controls are appropriately named. Event handling functions correctly. Fields clear appropriately. Data updates correctly after operations. Layout supports efficient use. **10 to >7.9 pts**	**Good** UI generally clear with minor layout or naming inconsistencies. Event handling mostly functional. Minor usability weaknesses. **7.9 to >6.4 pts**	**Satisfactory** Basic UI present and functional. Required controls included. Event handling works but may lack polish. Demonstrates minimum usability standard. **6.4 to >4.9 pts**	**Poo** UI unclear o inconsistent. handling unr Poor usabilit **4.9 to**
2.4 User Interface & Event Handling (10 marks)	**10 to >7.9 pts**	**7.9 to >6.4 pts**	**6.4 to >4.9 pts**	**4.9 to**

3.1 Test Case Table (5 marks)	**Excellent** At least eight well-structured test cases covering all CRUD operations, search/filter, validation rules and valid, invalid and boundary conditions. Expected and actual results are accurate and complete, with thorough functional coverage. **5 to >3.95 pts**	**Good** At least eight relevant test cases covering most required functionality and test conditions. Table is mostly complete and accurate, with minor coverage gaps or inconsistencies. **3.95 to >3.2 pts**	**Satisfactory** At least eight test cases with basic functional and validation coverage, but some required operations or conditions are insufficiently tested. Minor omissions or inaccuracies may be present. **3.2 to >2.45 pts**	**Poo** Fewer than e test cases, c substantially incomplete t Limited func coverage or unreliable re **2.45 to >**
3.2 Validation & Error Handling Evidence (5 marks)				

	Excellent	Good	Satisfactory	Poo
	At least two clear screenshots demonstrate validation messages. Supporting explanations, code excerpts or test results show that at least three validation/business rules are correctly enforced before database operations. Implementation is accurately explained.	Validation mostly enforced. Screenshots provided with minor gaps in explanation. Demonstrates sound understanding of input validation.	Basic validation evident. Screenshots included but limited explanation. Validation rules implemented but may be simplistic or inconsistently applied.	Limited valid evidence. Screenshots unclear or incomplete. explanation validation wo
3.3 Debugging Example (5 marks)	**Excellent** One clearly described genuine bug. Explanation includes cause, how it was identified, and how it was fixed. Demonstrates clear understanding of debugging process. **5 to >3.95 pts**	**Good** Bug described with mostly clear explanation of identification and resolution. Minor gaps in depth or clarity. **3.95 to >3.2 pts**	**Satisfactory** Bug described but explanation is basic. Cause or fix may lack detail. Demonstrates minimum understanding of debugging. **3.2 to >2.45 pts**	**Poo** Bug mentior poorly expla not clearly li the applicati **2.45 to >**

4.1 Functional Demonstration (10 marks)	**Excellent** Application runs correctly with a working database connection. All CRUD operations and the search/filter feature are demonstrated successfully. At least one validation rule is clearly shown working. Demonstration is smooth, logical and reliable.	**Good** Application runs and most CRUD operations function correctly. Search/filter and validation are demonstrated, with minor issues or omissions. Overall functionality is mostly reliable and clearly presented.	**Satisfactory** Application runs and basic CRUD functionality is demonstrated. Search/filter and validation are attempted, with some functional or demonstration weaknesses. Demonstrates basic understanding of the application's operation.	**Poo** Application r inconsistentl some CRUD features fail demonstrati Validation nc clearly demonstrate Significant functional weaknesses
	10 to >7.9 pts	**7.9 to >6.4 pts**	**6.4 to >4.9 pts**	**4.9 to**
4.2 Code Explanation & Understanding (5 marks)	**Excellent** Clearly explains custom classes, encapsulation, CRUD implementation, validation logic, and database connection. Responses are accurate and demonstrate strong foundational understanding.	**Good** Explains key components with mostly accurate understanding. Minor gaps in explanation or clarity.	**Satisfactory** Provides basic explanation of classes and functionality. Demonstrates minimum understanding but may lack depth or precision.	**Poo** Limited or ur explanation. Difficulty exp own code. Understandi appears wea
	5 to >3.95 pts	**3.95 to >3.2 pts**	**3.2 to >2.45 pts**	**2.45 to >**



Choose a file to upload

File permitted: DOCX

or

Canvas Files

I agree to the tool's **End-User License Agreement (https://api.turnitin.com/api/lti/1p0/user/static_eula)**
I have read, and acknowledge, that my submission complies with the terms of the Student Declaration.
*

Submit Assignment